Data Flow: "I'm planning a trip to Tokyo, what's the weather and time there?"

User | Browser (Streamlit UI) | Streamlit App (app.py) | PydanticAI Agent | LLM API (OpenAI-compatible) | travel_planner (Tool) | web_search (Tool) | get_datetime_info (Tool) | unit_converter (Tool)

Initialization (on app startup)

load_skills("SKILLS.md")

> Parses 6 skills:
> Web Search, Financial Calculator,
> Unit Converter, Text Summarizer,
> Date & Time, Travel Planner

build_skills_prompt(skills)

> Generates system prompt with
> skill names + instructions

Tool Registration (@agent.tool decorators)

@agent.tool web_search() →

> Registers function signature + docstring:
> **name:** "web_search"
> **params:** { "query": "string" }
> **desc:** "Search the web for current
> information on any topic."

@agent.tool calculate_growth() →

> **name:** "calculate_growth"
> **params:** { "initial": float,
> "rate": float, "years": int }
> **desc:** "Calculates compound interest
> growth. Pro members get a 5% bonus."

@agent.tool unit_converter() →

@agent.tool summarize_text() →

@agent.tool get_datetime_info() →

@agent.tool travel_planner() →

> **name:** "travel_planner"
> **params:** { "destination": "string",
> "home_timezone": int }
> **desc:** "Plan a trip by gathering
> weather, local time, and travel info."

Collects all 6 registered tools
into ToolDefinition list

> Each @agent.tool decorator converts
> the Python function into a
> **ToolDefinition** with:
> - name (function name)
> - description (docstring)
> - parameters_json_schema
> (auto-generated from type hints)
> This list is sent to the LLM as
> the **"tools"** field in every
> /chat/completions request.

User Input

Types: "I'm planning a trip to Tokyo,
what's the weather and time there?" →

st.chat_input() returns prompt →

Agent Invocation

deps = UserContext(
user_name="User",
is_pro_member=True
)

agent.run_sync(prompt, deps=deps) →

LLM Request #1: Decide which tool to call

POST /chat/completions →

> **Request:**
> system: "You are a helpful assistant.\n\n
> You have the following skills...\n
> 1. Travel Planner\n
> Help users plan trips by combining
> weather search, unit conversion,
> time zone lookup..."
> user: "I'm planning a trip to Tokyo,
> what's the weather and time there?"
> tools: [web_search, calculate_growth,
> unit_converter, summarize_text,
> get_datetime_info, travel_planner]

Tool call response ←

> **Response:**
> tool_calls: [{
> name: "travel_planner",
> arguments: {
> "destination": "Tokyo",
> "home_timezone": 8
> }
> }]

Tool Execution: travel_planner

travel_planner(
destination="Tokyo",
home_timezone=8
) →

Lookup timezone
tz_map["tokyo"] = 9

Simulated web_search(
"Tokyo weather travel conditions"
) →

"Clear skies, 25°C,
good travel conditions." ←

get local time
dest_tz=UTC+9, home_tz=UTC+8 →

dest: "2026-04-22 17:30 Wednesday"
home: "2026-04-22 16:30 Wednesday" ←

convert 25°C -> °F →

25.0°C = 77.0°F ←

> **Tool Result:**
> "🌍 Travel Briefing: Tokyo
> ━━━━━━━━━━━━━━━━
> 🌤️ Weather & Conditions:
> Clear skies, 25°C, good conditions
> 🕐 Time Zones:
> Tokyo: 2026-04-22 17:30 (UTC+9)
> Home: 2026-04-22 16:30 (UTC+8)
> Time difference: 1h ahead
> 🌡️ Temperature:
> 25.0°C = 77.0°F
> ✅ Travel Readiness: All checks passed!"

Returns unified briefing ←

LLM Request #2: Generate final response

POST /chat/completions →

> **Request:**
> messages: [
> ...previous messages...,
> tool_result: {
> name: "travel_planner",
> content: "🌍 Travel Briefing: Tokyo..."
> }
>]

Final text response ←

> **Response:**
> "Here's your travel briefing for Tokyo!
> The weather in Tokyo is currently clear
> with 25°C (77°F). Tokyo is 1 hour ahead
> of your home timezone.
> 🌍 Travel Briefing: Tokyo
> 🌤️ Clear skies, 25°C
> 🕐 Tokyo 17:30 (UTC+9) vs Home 16:30 (UTC+8)
> 🌡️ 25.0°C = 77.0°F
> ✅ All checks passed!"

Response Display

result.data = "Here's your travel briefing..." ←

st.session_state.messages.append(
{"role": "assistant",
"content": response_text}
)

st.markdown(response_text) ←

Displays formatted travel briefing ←

User | Browser (Streamlit UI) | Streamlit App (app.py) | PydanticAI Agent | LLM API (OpenAI-compatible) | travel_planner (Tool) | web_search (Tool) | get_datetime_info (Tool) | unit_converter (Tool)